# Milestone 3 Report

## ASCEND: Adaptive Self-supervised Continual Expert Network for Domain-Incremental Wi-Fi Sensing



| | |
|---|---|
| **Course** | ECE 310 Wireless Communication, Monsoon Semester 2026 |
| **Student** | Dhvanit Mahesuriya (AU2440039) |
| **Programme** | B.Tech Computer Science Engineering |
| **Domain** | Wi-Fi Sensing using CSI for Human Activity Recognition |
| **Category** | Research Project |
| **Date** | 31 October 2026 |



## 1. Summary


Wi-Fi sensing can recognise what a person is doing by looking at how their body changes the Wi-Fi signal. The signal measurement we use is called Channel State Information (CSI). It needs no camera and no wearable device.

There are two main problems. First, a model trained in one room usually works badly in another room. Second, if we keep training the same model on a new room, it forgets the old rooms. This is called *catastrophic forgetting*. Collecting labelled data in every new room also takes a lot of effort.

**Research question.** Can a Wi-Fi activity recognition model learn new rooms one after another, using few labels, without forgetting the old rooms?

**What this report covers.** Milestone 1 set up the problem and found the research gap. Milestone 2 showed the first ASCEND design and early results on MM-Fi. This report gives the final method and tests it on two datasets, MM-Fi and Widar3.0. It compares ASCEND with baselines, studies each component with an ablation, discusses what worked and what did not, and updates our position against the state of the art (SOTA).

**Main results** (single run, see Section 5):

- On MM-Fi, ASCEND reaches **90.30%** average final accuracy. SAMoE-C, the closest existing method, reaches **85.04%**. That is +5.26 points. Average forgetting drops from 7.27% to 1.19%.
- On Widar3.0, ASCEND reaches **66.75%** against 61.92% for SAMoE-C (+4.83 points).
- The smallest version, ASCEND-ULite, uses 35.4 MMACs. That is about 82% less compute than SAMoE-C (199.3 MMACs). It gives up 4.90 points of accuracy on MM-Fi.


### 1.1 Where each M3 objective is covered

| # | M3 objective | Where |
|---|---|---|
| 1 | Present the final method | Sections 2, 3 |
| 2 | Show the method works, with experiments | Sections 4, 5 |
| 3 | Compare with baselines using a clear protocol | Sections 4, 5 |
| 4 | Analyse results, including failures and limits | Sections 6, 8 |
| 5 | Final contribution and revised SOTA position | Section 7 |

## 2. How the Final Design Differs from M2

The M2 report described ASCEND in three phases and listed EWC and balanced replay among its five improvements. The final method is described as four modules, and forgetting is controlled by

freezing parts of the model. The table shows the differences. The MM-Fi results in this report are the same as the ones in M2.

| Item | M2 report | M3 (final) |
|---|---|---|
| Structure | Three phases (0, 1, 2) | Four modules: SSL pretraining, adapter, specialist, router |
| Backbone | Adapts to new rooms using EWC | Trained once on unlabelled $D_1$, then frozen for good |
| Forgetting control | EWC and balanced replay | Frozen backbone, adapters and specialists. EWC is used only as a baseline |
| Replay buffer | Ratio 0.15, equal share from every room | Ratio $\rho = 0.05$, used only to train the router |
| Router | Gate plus prototypes | Same: learned gate plus frozen SSL prototypes |
| SSL-KD and AIM masking | Proposed | Kept. AIM is trained with REINFORCE |
| Datasets | MM-Fi | MM-Fi and Widar3.0 |
| Domain order | One order | Standard order and one shuffled order |
| Evaluation | Accuracy and a synthetic check | Immediate and final accuracy, forgetting, MMACs, ablation |

**Planned in M2 but not done:** training with a fraction of the labels (label efficiency), repeated runs with several seeds, and an ablation study on MM-Fi. They are listed as limitations in Section 8.

# 3. Final Method: ASCEND

ASCEND is built for domain-incremental Wi-Fi activity recognition. Rooms $D_1, \ldots, D_N$ arrive one at a time. The set of activities $\mathcal{Y} = \{1, \ldots, K\}$ stays the same, but the CSI looks different in each room. The input is a CSI tensor $X \in \mathbb{R}^{C \times T \times S}$ (antennas × time × subcarriers). At test time we are *not* told which room a sample comes from.

## 3.1 Overview and frozen parameters

ASCEND keeps the shared part of the model separate from the room-specific parts (Fig. 1). The parameters are split as

$$\theta = \underbrace{\{\Phi\}}_{\text{frozen after Module 1}} \cup \bigcup_{k=1}^{N} \underbrace{\{A_k, E_k\}}_{\text{frozen after room } k}, \tag{1}$$

where $\Phi$ is the shared backbone, $A_k$ is the adapter of room $D_k$ and $E_k$ is its specialist. When a part finishes training, it is frozen and never changes again. Only the router $G$ keeps being updated when a new room arrives. Because old parts never change, a new room cannot overwrite what was learned for an old room.

## 3.2 Module 1: Self-supervised pretraining

The backbone $\Phi$ is trained once on *unlabelled* CSI from the first room $D_1$. It learns in two ways at the same time. It rebuilds hidden parts of the signal (masked reconstruction), and it makes the features of two differently masked views of the same sample agree (Barlow Twins):

$$\mathcal{L}_{\text{SSL}} = \mathcal{L}_{\text{rec}}(x, \hat{x}) + \beta_{BT}\, \mathcal{L}_{\text{BT}}(Z_1, Z_2), \tag{2}$$

$$\mathcal{L}_{\text{BT}} = \sum_i (1 - C_{ii})^2 + \lambda_{BT} \sum_i \sum_{j \neq i} C_{ij}^2. \tag{3}$$

Here $\mathcal{L}_{\text{rec}}$ is an L1 loss on the hidden positions, $Z_1$ and $Z_2$ are the features of the two masked views, and $C$ is the cross-correlation matrix between them. The first term of $\mathcal{L}_{\text{BT}}$ pushes matching feature

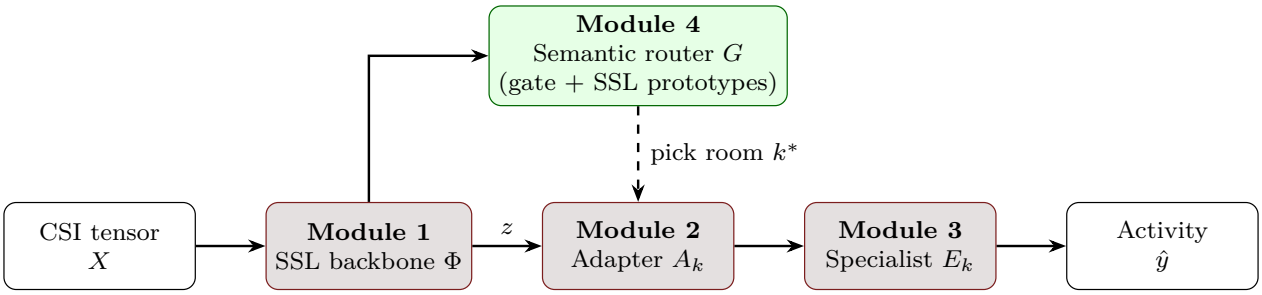



**Figure 1:** ASCEND at test time. The frozen backbone gives $z = \Phi(x)$. The router picks the room $k^*$, and only the matching adapter and specialist are used: $\hat{y} = E_{k^*}(A_{k^*}(z))$. Grey boxes are frozen once trained. The router (green) is the only part updated when a new room arrives.

dimensions to agree. The second term reduces repeated information between different dimensions. We use $\beta_{BT} = 0.2$, $\lambda_{BT} = 0.005$ and a mask ratio of 0.75.

**Adaptive Information Masking (AIM).** A small policy network gives a probability for each CSI patch. We sample which patches to hide from these probabilities. The policy is trained with REINFORCE, and its reward is how hard the hidden patches are to rebuild. So the mask learns to hide informative regions instead of using a fixed pattern. After training on $D_1$, the backbone is frozen and Module 1 is never run again.

### 3.3 Module 2: Domain-conditional adapter

For each new room $D_k$ we add a small bottleneck adapter with a residual connection:

$$Z_{\text{adapt}} = \Phi(X) + A_k(\Phi(X)). \tag{4}$$

The adapter only learns a small correction on top of the backbone features. It is first trained on *unlabelled* CSI of $D_k$ with the same two ideas as before, while the backbone stays frozen:

$$\mathcal{L}_{\text{adapt-SSL}} = \mathcal{L}_{\text{rec}}(X, \text{Dec}(Z^{(1)}_{\text{adapt}})) + \beta_{BT}\mathcal{L}_{\text{BT}}(Z^{(1)}_{\text{adapt}}, Z^{(2)}_{\text{adapt}}). \tag{5}$$

Here $\text{Dec}(\cdot)$ is the decoder that rebuilds the signal. Doing this step before using labels lets the features adjust to the new room first.

### 3.4 Module 3: Specialist expert

The specialist $E_k$ (convolution layers, a BiGRU and a classifier) predicts the activity from $Z_{\text{adapt}}$. It is trained with the usual cross-entropy loss plus a distillation term. The distillation term keeps its inner features close to the adapted features:

$$\mathcal{L}_{\text{KD}} = \left\|\text{Proj}(F_{\text{spec}}) - \text{sg}(Z_{\text{adapt}})\right\|_2^2, \qquad \mathcal{L}_{\text{spec}} = \mathcal{L}_{\text{CE}} + \alpha_{kd}\mathcal{L}_{\text{KD}}. \tag{6}$$

Here $F_{\text{spec}}$ is an inner specialist feature, Proj is a linear layer, $\text{sg}(\cdot)$ means no gradient flows through that term, and $\alpha_{kd}$ is tuned with Optuna. After training, $A_k$ and $E_k$ are frozen.

### 3.5 Module 4: Semantic router

At test time we do not know the room, so the router has to guess it. First it pools the backbone features $\{z_t\}_{t=1}^{T}$ over time with attention to get one context vector $c$:

$$e_t = v^\top \tanh(W z_t + b), \quad \alpha_t = \frac{\exp(e_t)}{\sum_{t'} \exp(e_{t'})}, \quad c = \sum_t \alpha_t z_t. \tag{7}$$

Then it mixes a learned gate $G(c)$ with the cosine similarity to frozen room prototypes $\mu_k$. A prototype is the average frozen-backbone feature of room $D_k$:

$$p = \text{Softmax}\big(\alpha\, G(c) + (1-\alpha)\,\tau\,\text{sim}(\bar{Z}, \mu)\big), \qquad k^* = \arg\max_j p_j. \tag{8}$$

**Algorithm 1** ASCEND training and inference

**Require:** Rooms $D_1, \ldots, D_N$ with unlabelled $X_k$ and labelled $(X_k, Y_k)$
Train $\Phi$ on masked views of $X_1$ with $\mathcal{L}_{\text{SSL}}$; freeze $\Phi$ ▷ Module 1, once
$\mathcal{B} \leftarrow \emptyset$
**for** each new room $D_k$ **do**
    Train adapter $A_k$ on unlabelled $X_k$ with $\mathcal{L}_{\text{adapt-SSL}}$ ▷ Module 2
    Train specialist $E_k$ on $(Z_{\text{adapt}}, Y_k)$ with $\mathcal{L}_{\text{spec}}$ ▷ Module 3
    Freeze $A_k$ and $E_k$
    Compute prototype $\mu_k$ from $\Phi(X_k)$
    Train router $G$ on $D_k \cup \mathcal{B}$; update $\mathcal{B}$ ▷ Module 4
**end for**
**Test time:** $z = \Phi(x)$; pick $k^*$ with $G$ and $\mu$; **return** $\hat{y} = E_{k^*}(A_{k^*}(z))$

**Table 1:** Configuration of the three ASCEND variants.

| Component | ASCEND | ASCEND-Lite | ASCEND-ULite |
|---|---|---|---|
| Backbone channels | 64, 96, 144, 256 | 48, 72, 108, 192 | 32, 48, 72, 128 |
| Adapter | 256→128→256 | 192→96→192 | 128→64→128 |
| Specialist channels | 128, 96, 64 | 96, 72, 48 | 128, 96, 64 |
| Specialist convolution | Standard | Standard | Depthwise-separable |
| Specialist GRU (layers / width) | 2 / 128 | 2 / 96 | 2 / 128 |
| Router blocks | 3 | 3 | 3 |
| Parameters | 4.4M | 2.5M | 2.5M |
| Inference cost | 214.2 MMACs | 121.7 MMACs | 35.4 MMACs |

Here $\bar{Z}$ is the backbone feature averaged over time, $\alpha \in [0, 1]$ balances the two scores, and $\tau = 5.0$. The prototypes never change, so they give the router steady reference points even when there is little data from old rooms. The gate is trained to predict the room, using the current room plus a small replay buffer $\mathcal{B}$ that holds a fraction $\rho = 0.05$ of past data:

$$\mathcal{L}_{\text{router}} = \mathcal{L}_{\text{CE}}(\hat{d}, d), \qquad (X, d) \in D_k \cup \mathcal{B}. \tag{9}$$

### 3.6 Training and inference

### 3.7 Model variants

We built three sizes. They have the same structure, training steps and router, and differ only in size (Table 1). ASCEND-Lite uses fewer channels. ASCEND-ULite also uses depthwise-separable convolutions in the specialist.

## 4. Experimental Setup

### 4.1 Datasets

- **MM-Fi** [10]. Recorded with a TP-Link N750 router (802.11n). It has four rooms (two living rooms and two meeting rooms), used as $D_1$ to $D_4$ in that order. Each room has ten subjects and $K = 27$ activities. The CSI shape is $3 \times 10 \times 114$ (antennas × time × subcarriers).
- **Widar3.0** [11]. Recorded with Intel 5300 network cards. Four capture sessions are used as the four domains, with $K = 6$ gestures. The raw CSI is $3 \times 500 \times 30$. We average-pool it along time to $3 \times 10 \times 30$. We use one receiver per sample, so the same recording never appears twice through different receivers.

Both datasets use an 80/20 train–test split. They differ in hardware, number of classes and amount

of data, so using both tests how well the method carries over to a different setting. MM-Fi is also the benchmark used by SAMoE-C [4] and by the EWC baseline [5].

### 4.2 Baselines

Five single-model baselines are trained on the rooms one after another: AutoFi [6], CAPC [7], ResNet-18 [9], MaskCAE [8], and a CNN-ABLSTM with an EWC penalty [5]. The first four are simply fine-tuned on each new room. The fifth adds the EWC penalty to slow forgetting. The continual-learning baseline is SAMoE-C [4]. It is a mixture-of-experts method whose backbone is trained with labels on the first room. All baselines use the hyperparameters from their original papers.

### 4.3 Assumptions

- **Domain-incremental setting.** Rooms arrive one at a time. The activities are the same in every room, and only the CSI distribution changes.
- **No room label at test time.** The router has to work out the room.
- **Shared structure across rooms.** We assume the signal structure learned from unlabelled $D_1$ is useful enough that a small adapter can bridge the gap to later rooms.
- **Data in each room.** Unlabelled CSI is used to train the adapter. Labelled CSI is used to train the specialist.
- **Small replay buffer.** Only a small share of past data ($\rho = 0.05$) is kept, and it is used only to train the router.
- **One person.** Each sample contains one person. On Widar3.0 we use one receiver per sample.
- **Split inside each room.** The 80/20 split is made inside each room, so we do not test on new subjects.

### 4.4 Evaluation protocol and metrics

Rooms are trained in order. After each room, we test on all rooms seen so far. Let $a_{j,k}$ be the test accuracy on room $k$ after training up to room $j$.

- **Immediate accuracy:** $a_{k,k}$, measured right after learning room $k$.
- **Final accuracy:** $a_{N,k}$, measured after all rooms are learned. Its average over rooms, $\frac{1}{N}\sum_k a_{N,k}$, is our main metric.
- **Forgetting:** $F_k = a_{k,k} - a_{N,k}$, averaged over rooms. Lower is better.
- **Cost:** number of parameters and inference cost in MMACs.

To check sensitivity to room order, we also run ASCEND on MM-Fi in the order $D_3 \to D_1 \to D_2 \to D_4$.

### 4.5 Training settings and hardware

The structural settings are the same for every room. Three sensitive settings are tuned with Optuna [12]: $\alpha_{kd}$, the specialist learning rate and the router learning rate. Widar3.0 gets 50 trials instead of 5 because it was more sensitive when each room has little data. All runs used one NVIDIA Quadro RTX 5000 GPU with 16 GB of memory.

## 5. Results

All numbers come from a single run unless stated otherwise.

**Table 2:** Training settings for ASCEND.

| Setting | MM-Fi | Widar3.0 |
|---|---|---|
| Optimizer | AdamW | AdamW |
| Module 1: learning rate / batch size | $10^{-4}$ / 64 | $10^{-4}$ / 64 |
| Module 2: epochs | 50 | 50 |
| Module 3: learning rate / batch size / epochs | $3 \times 10^{-4}$ / 32 / 35 | $1.30 \times 10^{-4}$ / 64 / 35 |
| Module 4: epochs | 20 | 20 |
| Replay fraction $\rho$ | 0.05 | 0.05 |
| Optuna trials | 5 | 50 |
| Classes $K$ | 27 | 6 |
| Input shape | $3 \times 10 \times 114$ | $3 \times 10 \times 30$ |
| Train–test split | 80/20 | 80/20 |

**Table 3:** Final accuracy (%) on MM-Fi after training on $D_1 \rightarrow D_4$. Bold marks the best value in each column among the continual-learning frameworks.

| Method | D1 | D2 | D3 | D4 | Avg. |
|---|---|---|---|---|---|
| *Single-model baselines (trained room by room)* | | | | | |
| AutoFi | 3.46 | 4.25 | 4.85 | 67.00 | 19.89 |
| ResNet-18 | 4.10 | 5.14 | 7.41 | 82.54 | 24.80 |
| CAPC | 3.60 | 4.17 | 4.16 | 57.89 | 17.45 |
| MaskCAE | 4.38 | 4.50 | 4.18 | 55.14 | 17.05 |
| CNN-ABLSTM + EWC | 66.60 | 56.74 | 49.98 | 57.56 | 57.72 |
| *Continual-learning frameworks* | | | | | |
| SAMoE-C | 90.77 | 83.32 | 77.82 | **88.23** | 85.04 |
| **ASCEND** | **94.21** | **89.96** | **89.13** | 87.92 | **90.30** |
| ASCEND-Lite | 91.28 | 83.74 | 76.63 | 80.68 | 83.08 |
| ASCEND-ULite | 85.80 | 79.81 | 78.12 | 76.85 | 80.14 |

## 5.1 Single models forget the earlier rooms

Table 3 shows final accuracy on MM-Fi after all four rooms. AutoFi, ResNet-18, CAPC and MaskCAE keep only 3.5% to 7.4% accuracy on $D_1$ to $D_3$. They do well only on the last room, $D_4$.

This is not because the models are too small. ResNet-18 reaches 90.85% on $D_1$ when it is trained on that room alone. After training on $D_2$ to $D_4$, it drops to 4.10%. Every weight is shared, so later rooms overwrite what was learned earlier. EWC helps (57.72% average) but does not solve the problem.

## 5.2 ASCEND versus SAMoE-C on MM-Fi

ASCEND raises the average final accuracy from 85.04% to 90.30% (+5.26 points). The gains by room are +3.44 on $D_1$, +6.64 on $D_2$ and +11.31 on $D_3$. SAMoE-C is 0.31 points better on $D_4$.

Table 4 also shows accuracy right after learning each room, and forgetting. Average forgetting falls from 7.27% to 1.19%, about six times lower.

**Room order.** With the shuffled order $D_3 \rightarrow D_1 \rightarrow D_2 \rightarrow D_4$, ASCEND reaches 90.28% average accuracy. This is only 0.02 points away from the normal order. We did not run SAMoE-C on the shuffled order.

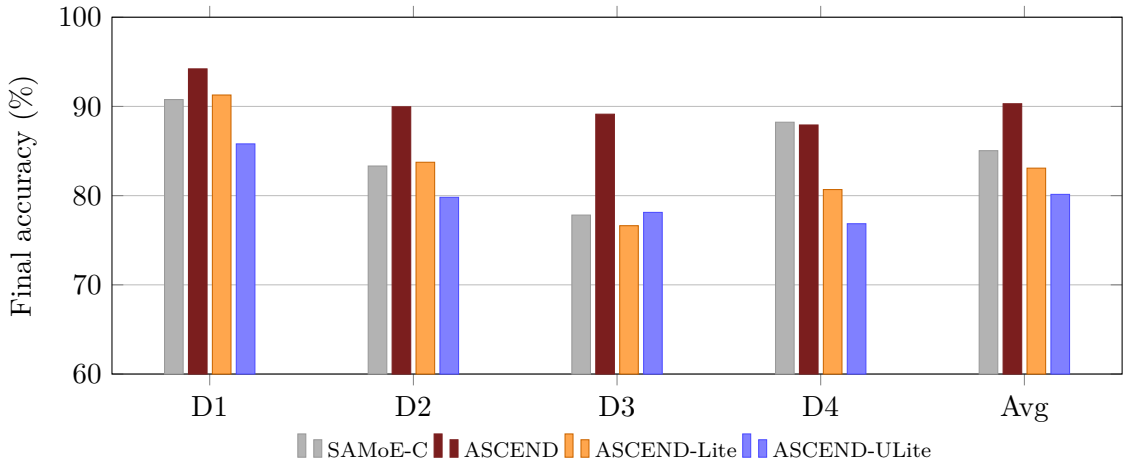



**Figure 2:** Final accuracy per room for the continual-learning frameworks on MM-Fi. The vertical axis starts at 60%.

**Table 4:** Immediate accuracy, final accuracy and forgetting (%) on MM-Fi.

| Method | Stage | D1 | D2 | D3 | D4 | Avg. |
|---|---|---|---|---|---|---|
| SAMoE-C | Immediate | 96.27 | 93.42 | 91.27 | 88.28 | 92.31 |
| | Final | 90.77 | 83.32 | 77.82 | 88.23 | 85.04 |
| | Forgetting | 5.50 | 10.10 | 13.45 | 0.05 | 7.27 |
| ASCEND | Immediate | 95.18 | 92.90 | 89.91 | 87.97 | 91.49 |
| | Final | 94.21 | 89.96 | 89.13 | 87.92 | 90.30 |
| | Forgetting | 0.97 | 2.94 | 0.78 | 0.05 | 1.19 |
| ASCEND-Lite | Immediate | 91.71 | 87.11 | 82.57 | 80.77 | 85.54 |
| | Final | 91.28 | 83.74 | 76.63 | 80.68 | 83.08 |
| | Forgetting | 0.43 | 3.37 | 5.94 | 0.09 | 2.46 |
| ASCEND-ULite | Immediate | 88.59 | 83.18 | 80.52 | 77.10 | 82.35 |
| | Final | 85.80 | 79.81 | 78.12 | 76.85 | 80.14 |
| | Forgetting | 2.79 | 3.37 | 2.40 | 0.25 | 2.20 |

## 5.3 Second dataset: Widar3.0

Table 5 shows that the gain also appears on different hardware and a different set of activities. ASCEND reaches 66.75% average final accuracy against 61.92% for SAMoE-C (+4.83 points). The biggest gap is on $D_2$: 66.67% against 52.67% (+14.00 points).

Here both methods forget very little (0.51% for ASCEND and 0.39% for SAMoE-C). So the accuracy gap does *not* come from better forgetting control. ASCEND is already ahead right after learning $D_2$ (67.78% against 54.22%). This suggests better features, not better memory. ASCEND-ULite has zero forgetting on all four rooms.

## 5.4 Accuracy versus compute

Figure 3 plots MM-Fi accuracy against inference cost. Full ASCEND uses 214.2 MMACs, about 7.5% more than SAMoE-C (199.3 MMACs), and gains 5.26 points. ASCEND-Lite uses 121.7 MMACs (39% less than SAMoE-C) and is 1.96 points below it. ASCEND-ULite uses 35.4 MMACs (82% less) and is 4.90 points below it. All three are far above the single-model baselines.

**Table 5:** Immediate accuracy, final accuracy and forgetting (%) on Widar3.0.

| Method | Stage | D1 | D2 | D3 | D4 | Avg. |
|---|---|---|---|---|---|---|
| SAMoE-C | Immediate | 71.78 | 54.22 | 62.28 | 60.95 | 62.31 |
| | Final | 71.78 | 52.67 | 62.28 | 60.95 | 61.92 |
| | Forgetting | 0.00 | 1.56 | 0.00 | 0.00 | 0.39 |
| ASCEND | Immediate | 69.67 | 67.78 | 67.43 | 64.19 | 67.27 |
| | Final | 69.11 | 66.67 | 67.05 | 64.19 | 66.75 |
| | Forgetting | 0.56 | 1.11 | 0.38 | 0.00 | 0.51 |
| ASCEND-Lite | Immediate | 66.22 | 60.67 | 65.52 | 55.81 | 62.05 |
| | Final | 66.22 | 58.89 | 65.52 | 55.81 | 61.61 |
| | Forgetting | 0.00 | 1.78 | 0.00 | 0.00 | 0.44 |
| ASCEND-ULite | Immediate | 62.78 | 55.11 | 61.52 | 61.14 | 60.14 |
| | Final | 62.78 | 55.11 | 61.52 | 61.14 | 60.14 |
| | Forgetting | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 |

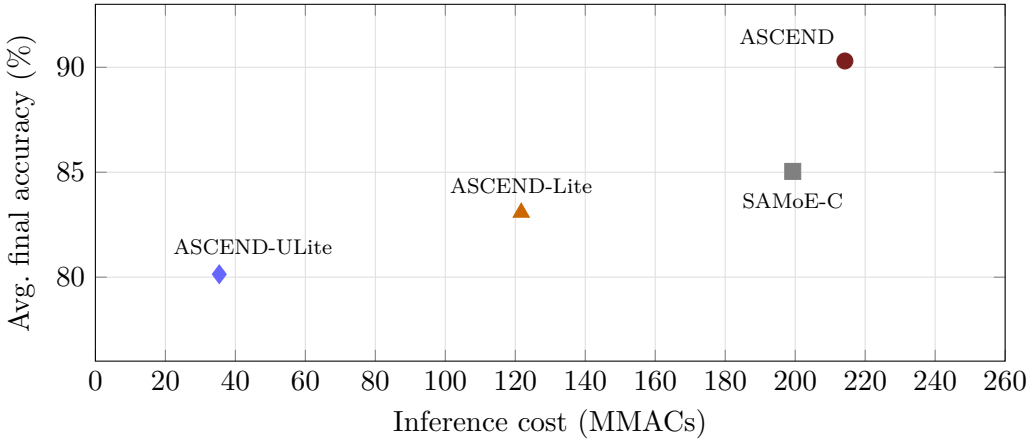



**Figure 3:** Accuracy versus compute on MM-Fi.

### 5.5 Ablation study (Widar3.0)

We removed one component at a time and measured the average final accuracy (Table 6, Fig. 4). Every removal lowers accuracy. The prototype router matters most (−10.24 points), then SSL pretraining (−9.74) and the domain adapter (−8.12). Knowledge distillation (−4.37) and AIM masking (−4.17) matter less.

## 6. Critical Analysis

### 6.1 What worked

- **Old rooms are kept well.** The biggest MM-Fi gain is on $D_3$ (+11.31 points). $D_3$ is a meeting room, while $D_1$ is a living room. Freezing the backbone and giving each room its own adapter and specialist protects what was learned.
- **The gain holds on both datasets.** ASCEND beats SAMoE-C by +5.26 points on MM-Fi and +4.83 on Widar3.0, even though the hardware, activities and data size differ.
- **Every component helps.** Each ablation costs at least 4 points on Widar3.0.
- **Room order matters little.** The shuffled MM-Fi order changes the average by only 0.02 points.

**Table 6:** Ablation study on Widar3.0 (average final accuracy).

| Configuration | Avg. accuracy (%) | Change |
|---|---|---|
| Full ASCEND | 66.75 | – |
| Without SSL prototype router | 56.51 | −10.24 |
| Without SSL pretraining | 57.01 | −9.74 |
| Without domain adapter | 58.63 | −8.12 |
| Without knowledge distillation | 62.38 | −4.37 |
| Without AIM (random masking) | 62.58 | −4.17 |

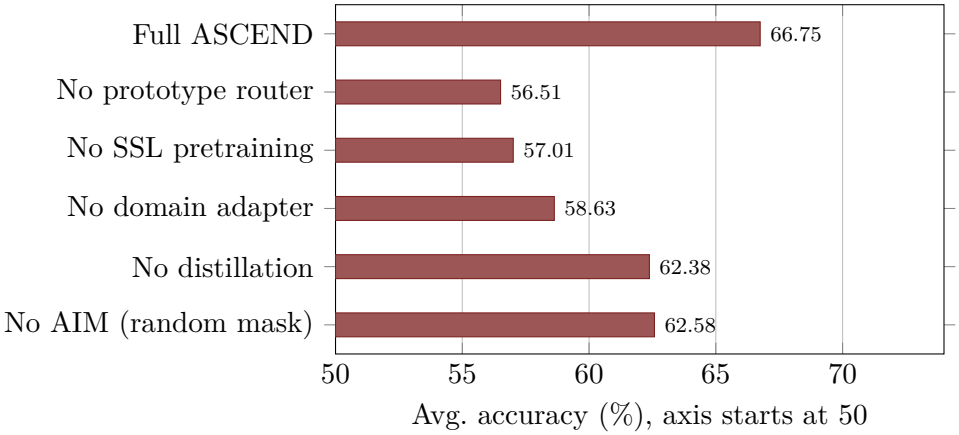



**Figure 4:** Ablation on Widar3.0.

## 6.2 What did not work well

- **Lower accuracy right after learning a room (MM-Fi).** ASCEND averages 91.49% against 92.31% for SAMoE-C. SAMoE-C is also slightly better on $D_4$ (88.23% against 87.92%). So on MM-Fi, ASCEND wins by *remembering*, not by learning each room better.
- **Widar3.0, room $D_1$.** SAMoE-C is better here (71.78% against 69.11%, a gap of 2.67 points).
- **The small variants lose accuracy.** On MM-Fi, ASCEND-Lite (83.08%) and ASCEND-ULite (80.14%) are below SAMoE-C (85.04%). On Widar3.0 they are 0.31 and 1.78 points below. They save compute but do not beat SAMoE-C. Full ASCEND uses about 7.5% more compute than SAMoE-C.
- **Low accuracy on Widar3.0.** About 67% on a six-class task is still low, even though it is better than SAMoE-C.
- **Forgetting is not zero.** The backbone, adapters and specialists are frozen. So the small amount of forgetting that remains can only come from the router sending an old-room sample to the wrong room. We did not measure routing accuracy, so this is our reasoning and not a measured result.

## 6.3 Possible explanations

These are our best explanations. We have not tested them directly.

- **The backbone is trained by reconstruction.** To rebuild the signal, the backbone must keep its full shape, including room-specific changes that a classifier trained only for activity labels may throw away. This could explain the bigger gains on rooms that differ a lot from $D_1$. On Widar3.0, a t-SNE plot shows the sessions forming separate clusters in the frozen feature space, which fits this idea. To test it properly, we would train a labelled backbone inside the same pipeline and compare.

- **The router.** Removing the prototype router hurts most. A likely reason is that the learned gate alone confuses similar rooms when there is little data, while the fixed prototypes give it steady reference points.
- **The source of the gain is different on each dataset.** On MM-Fi, SAMoE-C forgets a lot (7.27%) and ASCEND does not, so the gain is mostly about remembering. On Widar3.0 both forget little, and ASCEND is already ahead right after learning each room, so the gain is mostly about better features.

### 6.4 Things that limit how far the results can be trusted

- **One run only.** We have one run per setting, so there are no error bars. Differences of a few tenths of a point, like the $D_4$ gap on MM-Fi, should not be read as real.
- **Tuning is uneven.** We tuned ASCEND with Optuna (5 trials on MM-Fi, 50 on Widar3.0). The baselines use their published settings. Some of the gain may come from this extra tuning.
- **Ablation on one dataset.** We ran the ablation only on Widar3.0. We do not know the effect sizes on MM-Fi.
- **Same subjects in train and test.** The 80/20 split is made inside each room, so we make no claim about new people.
- **Not completely replay-free.** The router uses a small replay buffer ($\rho = 0.05$). This reduces, but does not remove, the storage and privacy concern.

## 7. Revised SOTA Position and Final Contributions

### 7.1 Comparison with related work

| Work | Main idea | SSL | Cont. | Forget. | Note |
|---|---|---|---|---|---|
| Wi-DMAR [2] | Conditional diffusion for cross-domain HAR | No | No | No | One-time domain shift |
| CIG-MAE [3] | Cross-modal masked autoencoder | Yes | No | No | Fewer labels, not continual |
| ConSense [1] | Continual sensing by growing and picking | No | Yes | Yes | Continual, supervised |
| SAMoE-C [4] | Supervised backbone, router, one expert per room | No | Yes | Yes | Closest work |
| **ASCEND** | Frozen SSL backbone, adapters, specialists, hybrid router | **Yes** | **Yes** | **Yes** | Tested on MM-Fi and Widar3.0 |

"Cont." means continual learning. "Forget." means forgetting is handled.

**Revised position.** In M2 we only said that ASCEND aims to fill a gap. Now we have evidence. Among the works we reviewed, ASCEND is the one that combines Wi-Fi CSI sensing, learning rooms one by one, self-supervised pretraining and forgetting control by freezing. Under our protocol it beats SAMoE-C, the closest continual method, on both datasets. We do not claim to be the overall state of the art. The claim is limited to this comparison on two datasets, from single runs. The accuracy gain comes mainly from better memory on MM-Fi and from better features on Widar3.0. Only the smaller variants use less compute than SAMoE-C, and they give up some accuracy.

### 7.2 Final contributions

1. **A self-supervised continual expert model for Wi-Fi sensing.** A backbone is trained once on unlabelled CSI and frozen. Each new room adds one small adapter and one specialist, so later rooms cannot overwrite earlier ones.

2. **Adaptive masking (AIM).** A policy trained with REINFORCE chooses which CSI patches to hide. It has the smallest effect of the five components in our ablation (−4.17 points).
3. **Two-step room adaptation and a hybrid router.** The adapter is first trained without labels, then the specialist is trained with labels and distillation. The router combines a learned gate with frozen SSL prototypes. Removing it costs the most accuracy (−10.24 points).
4. **Evidence on two datasets.** We report accuracy, forgetting, compute, an ablation and a shuffled-order check on MM-Fi and Widar3.0.

## 8. Limitations and Future Work

- **Storage grows with the number of rooms.** Each room adds one adapter and one specialist. Compressing adapters or sharing them between similar rooms could limit this.
- **One person only.** We have not tested rooms where several people move at once.
- **The router only knows the rooms it has seen.** It always picks one of them and cannot say "this is a new room."
- **The backbone depends on $D_1$.** It is trained on unlabelled data from the first room only. We tested this with one shuffled order.
- **Not done yet:** runs with several seeds and error bars, a test with fewer labels, an ablation on MM-Fi, a measurement of routing accuracy, and SAMoE-C on the shuffled order.

## 9. Project Status

| Area | Status | Outcome |
|---|---|---|
| Problem and literature gap | Done | Domain-incremental Wi-Fi HAR with few labels and forgetting |
| ASCEND implementation | Done | Four modules, three sizes |
| Baseline comparison | Done | Five single-model baselines and SAMoE-C on two datasets |
| Ablation study | Done (Widar3.0 only) | Five components removed one at a time |
| Room-order check | Partly done | One shuffled order, ASCEND only |
| Few-label test, several seeds | Not done | Listed as future work |
| Paper draft | Written | Extends this report |

## 10. Conclusion

ASCEND combines a frozen self-supervised backbone, one adapter and specialist per room, and a hybrid router for Wi-Fi sensing in rooms that arrive one at a time. Single models trained room by room keep under 8% accuracy on the earlier MM-Fi rooms. Full ASCEND keeps between 87.9% and 94.2% on every room. Compared with SAMoE-C, it gains +5.26 points on MM-Fi and +4.83 points on Widar3.0. On MM-Fi the gain comes mostly from much lower forgetting (1.19% against 7.27%). On Widar3.0 it comes mostly from better features.

The results also show limits. ASCEND is slightly less accurate right after learning each MM-Fi room, the smaller variants trail SAMoE-C in accuracy, all results come from single runs, and the router still needs a small replay buffer. The next steps for the final paper are runs with several seeds, a test with fewer labels, and an ablation on MM-Fi.

# References

[1] R. Li, T. Deng, S. Feng, M. Sun, and J. Jia, "ConSense: Continually sensing human activity with WiFi via growing and picking," in *Proc. AAAI Conf. Artif. Intell.*, vol. 39, pp. 14292–14300, 2025.

[2] C. Tang, P. Tang, H. Zhang, J. Jin, and S. Mao, "Wi-DMAR: Cross-domain human activity recognition via an enhanced conditional diffusion model," *IEEE Internet Things J.*, vol. 13, no. 11, pp. 25343–25353, 2026.

[3] G. Liu, Y. Hao, and Y. Zou, "CIG-MAE: Cross-modal information-guided masked autoencoder for self-supervised WiFi sensing," *IEEE Internet Things J.*, 2026, doi: 10.1109/JIOT.2026.3696504.

[4] W. Zheng, Y. Mao, and I. W.-H. Ho, "Scene-adaptive continual learning for CSI-based human activity recognition with mixture of experts," arXiv:2605.06447, 2026.

[5] Z. He *et al.*, "Robust cross-scenario WiFi wireless sensing using incremental learning and elastic weight consolidation loss," *IEEE Internet Things J.*, vol. 12, no. 13, pp. 24288–24299, 2025.

[6] J. Yang *et al.*, "AutoFi: Toward automatic Wi-Fi human sensing via geometric self-supervised learning," *IEEE Internet Things J.*, vol. 10, no. 8, pp. 7416–7425, 2023.

[7] B. Barahimi *et al.*, "Context-aware predictive coding: A representation learning framework for WiFi sensing," *IEEE Open J. Commun. Soc.*, vol. 5, pp. 6119–6134, 2024.

[8] D. Cheng *et al.*, "MaskCAE: Masked convolutional autoencoder via sensor data reconstruction for self-supervised human activity recognition," *IEEE J. Biomed. Health Inform.*, vol. 28, pp. 2687–2698, 2024.

[9] K. He *et al.*, "Deep residual learning for image recognition," in *Proc. CVPR*, 2016, pp. 770–778.

[10] J. Yang *et al.*, "MM-Fi: Multi-modal non-intrusive 4D human dataset for versatile wireless sensing," in *Proc. NeurIPS*, 2023.

[11] Y. Zhang *et al.*, "Widar3.0: Zero-effort cross-domain gesture recognition with Wi-Fi," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 11, pp. 8671–8688, 2022.

[12] T. Akiba *et al.*, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. KDD*, 2019, pp. 2623–2631.